

固定接管提示词(两套,保存到你的备忘录,永不过时)

设计原理(为什么这两段可以"固定"):你发送的只是**指针**——真正的规程存在仓库里

(docs\两份接管提示词,git 管理,流程变化时由"收尾五联动"负责更新它们)。

指针里只有三样永不变的东西:本地路径、GitHub 仓库地址、规程文件名。

所以哪怕三周后流程大改,你发的这两段依然有效——AI 读到的永远是最新规程。

套 A:新 Claude Code 账号全面接管(开发全权)

直接复制发送,不用改一个字:

你接管 F:\abb_wh_work 的「智储优控」比赛项目(2026 ABB 杯赛道一,立体仓储仓位优化)。
若该目录不存在或为空:先 `git clone https://github.com/7shangguanqianer-gif/FishAndCat.git`
到 F:\abb_wh_work 再继续。

第一步:读 docs\ClaudeCode 接管提示词.md,严格执行其中的「接手三步」与全部铁律(含任务收尾五联动);

自检:若 现状与任务.md §5 的内容与 `git log` 最新几条提交明显不符(说明上个会话没走完收尾),以 `git log` 与文件实物为准,先修复 §5 再开工,并向我报告差异。

完成接手三步后:①先自答接管提示词里的 5 道「接管测验」(我会肉眼核对,答不对就重读)
②汇报你看到的项目状态(一段话)③按 §5 待办顺序开始第一个任务。

套 C:Codex 日常派活(最常用,一句话)

Claude 会把任务批次写进信箱文件,你只需对 Codex 说这一句(永远不变):

读 F:\abb_wh_work_通信\致 Codex.md 并执行,结果写进同目录 致 Claude.md

Codex 干完后,你对 Claude 说"Codex 交了"即可——Claude 自己去读汇报信箱,

不需要你复制粘贴任何内容。这就是"Claude↔Codex 对话"的落地方式(异步信箱)。

套 B: Codex 执行任务(临时指派,不走信箱时用)

直接复制发送,把最后一行换成你要它干的活:

```
你是 F:\abb_wh_work「智储优控」项目的执行工程师,只执行不设计。  
第一步:读 docs\Codex 总对接提示词.md 并严格遵守其中全部铁律;  
做完任务必须走提示词里的「收尾五联动」,一步不许跳。  
若对任务与项目现状有冲突疑问:以 现状与任务.md §5 为准,仍不清楚就停下来问,不许猜。  
  
本次任务:【在这里写你要它干的具体活,一句话也行】
```

诚实的保证边界(什么能保证,什么不能)

能保证的(结构性,已验证):

1. **入口不过时**:上面两段只含稳定事实,规程本体在仓库里随流程演进(五联动维护);
2. **状态可信**:接手三步强制读单一真相源+跑 `sim\run_all.py`(回归+实验+头条+doc_check,

实测 29 秒全绿)——接手者第一时间就知道底盘是否完好;

3. **坏状态可发现**:回归测试锁死口径与头条数字、doc_check 锁死文档联动——

谁改坏了,红灯立刻亮,规程要求"红了停下报告,不许自修";

4. **任何破坏可回滚**:git 全历史+GitHub 云端,最坏情况回退到上一个绿灯提交。

不能绝对保证的:AI 是否 100% 遵守规程是概率问题(就像我也犯过执行单没更新的错)。

但错误"不被发现地积累"这条路已被堵死——三道硬防线(红绿灯工具、git 回滚、

你每周五的验收清单人工核对)保证错误最多存活一个检查周期。

这是工程上能承诺的"无误":**不是不犯错,是错误必然暴露、必然可撤销。**